Transactions

198:441/541 Distributed and Cloud Computing
Lecture 16

Instructor:
Michael A. Palis
palis@camden.rutgers.edu

Transaction

- **Series of operations executed by client**
  - Each operation is an RPC to a server
- **Transactions are created and managed by a coordinator, which implements the following interface:**
  - **openTransaction() ⇒ TID;**
    - *starts a new transaction and delivers a unique transaction ID* **TID**. *This identifier will be used in the other operations in the transaction*
  - **closeTransaction(TID) ⇒ (commit, abort);**
    - *ends a transaction: a* **commit** *return value indicates that the transaction has committed; an* ***abort*** *return value indicates that it has aborted*
  - **abortTransaction(TID);**
    - *aborts the transaction; can be executed by the server or the client at any time*

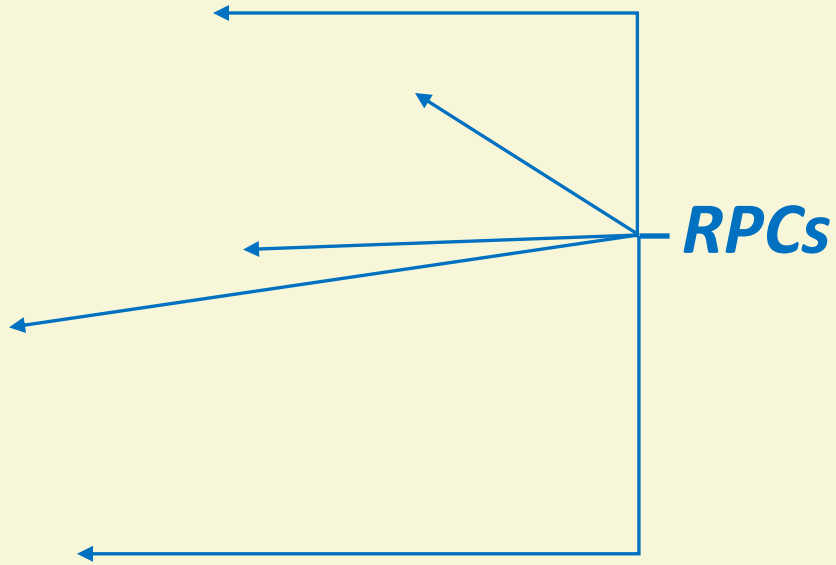
Transaction, Cont.

- **Commit**
  - All changes requested by the transaction are permanently recorded on the server
  - All future transactions that access the same data will see the results of all the changes made during the transaction
- **Abort**
  - None of the changes made during the transaction are recorded
  - None of the effects of this transaction will be visible to future transactions, either in the objects or in their copies in permanent storage

Transaction: Example

- **Client code**
  - Client sends RPCs to server that holds data
  - Server executes RPCs and returns results to client

```
int transaction_id = openTransaction();
x = server.getFlightAvailability(ABC, 123, date);
if (x > 0) {
  y = server.bookTicket(ABC, 123, date);
  server.putSeat(y, "aisle");
}
// commit entire transaction or abort
closeTransaction(transaction_id);
```

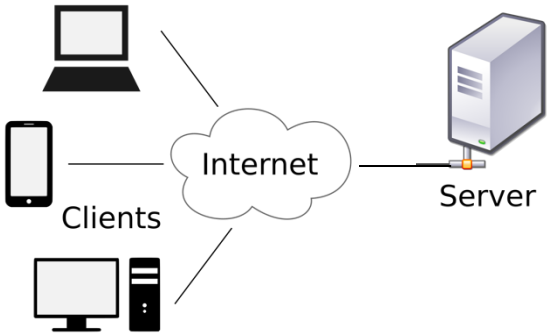


ACID Properties for Transactions

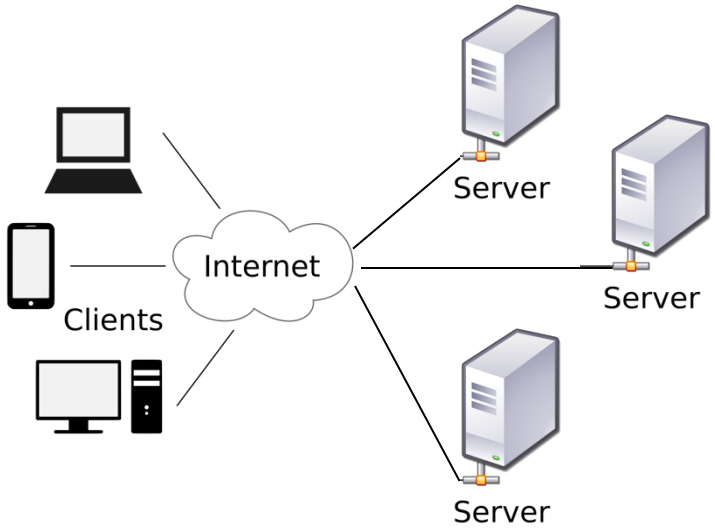
- **Atomicity**
  - All or nothing
  - Each transaction either executes completely or not at all
- **Consistency**
  - If the server starts in a consistent state, the transaction ends with the server in a consistent state
- **Isolation**
  - Multiple transactions do not interfere with each other
  - No access to intermediate results/states of other transactions
- **Durability**
  - After a transaction has completed successfully, all its effects are preserved (e.g., stored in permanent storage)

Concurrency Control

- **Goal: manage the concurrent execution of multiple transactions while preserving ACID properties**
  - Problems arise even with a single server
  - Problems compounded with multiple servers (distributed transactions)
- **Will study single server case first**



Single Server



Multiple Servers

Multiple Clients, One Server

Time

Transaction T1

```
x = getSeats(ABC123);

if (x > 0) {
  x = x - 1;
  write(x, ABC123);
}

commit
```

Transaction T2

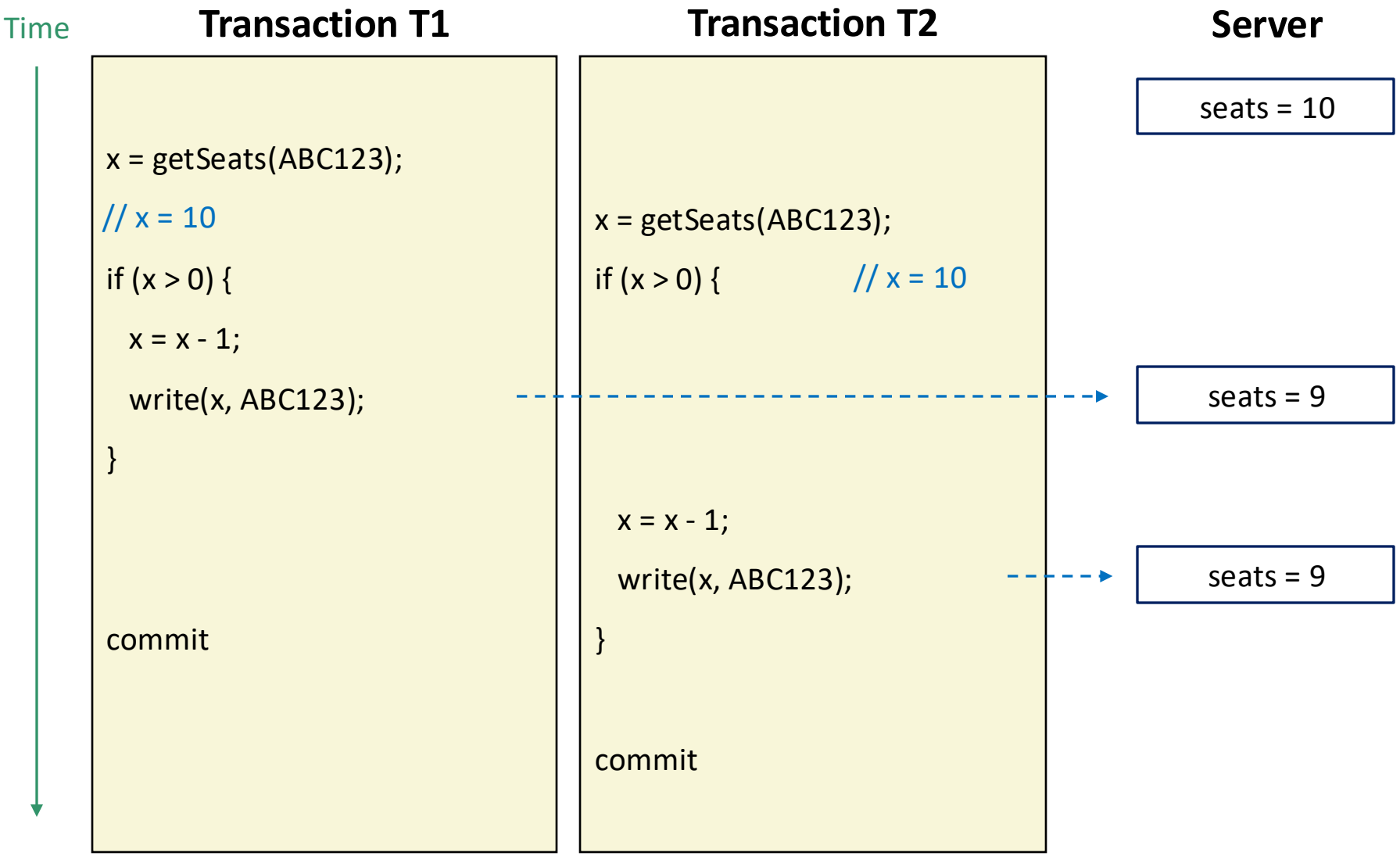
```
x = getSeats(ABC123);
if (x > 0) {

  x = x - 1;
  write(x, ABC123);
}

commit
```

Server

seats = 10

What could go wrong?

Lost Update Problem



Lost Update Problem

Time

Transaction T1	Transaction T2	Server
		seats = 10
x = getSeats(ABC123);	x = getSeats(ABC123);	
// x = 10	if (x > 0) { // x = 10	**T1's or T2's update was lost!**
if (x > 0) {		
x = x - 1;		seats = 9
write(x, ABC123);		
}		
	x = x - 1;	seats = 9
	write(x, ABC123);	
commit	}	
	commit	

Inconsistent Retrieval Problem

Time

Transaction T1	Transaction T2
x = getSeats(ABC123);	
y = getSeats(ABC789);	
write(x-5, ABC123);	
// ABC123 = 5	
	x = getSeats(ABC123);
	y = getSeats(ABC789);
	// x = 5, y = 15
write(y+5, ABC789);	
	print("Total:", x+y);
	// Prints "Total: 20"
commit	
	commit

Server

ABC123 = 10
ABC789 = 15

Inconsistent Retrieval Problem

Time

Transaction T1	Transaction T2
x = getSeats(ABC123);	
y = getSeats(ABC789);	
write(x-5, ABC123);	
// ABC123 = 5	
	x = getSeats(ABC123);
write(y+5, ABC789);	y = getSeats(ABC789);
	// x = 5, y = 15
	print("Total:", x+y);
commit	// Prints "Total: 20"
	commit

Server

ABC123 = 10
ABC789 = 15

T2's sum is wrong! Should be "Total: 25"

Multiple Clients, One Server

- **What could go wrong?**
  - Lost updates
  - Inconsistent retrievals
- **How to prevent transactions from multiple clients from affecting each other?**

Concurrent Transactions

- **To prevent transactions from affecting each other**
  - Serial execution: execute transactions one at a time at server
    - *Enforce mutual exclusion*
    - *E.g., grab a global lock before executing a transaction; release the lock only after the transaction has committed (or aborted)*
  - But this reduces number of concurrent transactions
  - *Transactions per second* directly related to revenue of companies
    - *This metric needs to be maximized*
- **How do we increase concurrency while maintaining correctness (ACID)?**
  - Execute transactions concurrently in such a way that end-result is the same as if the transactions were executed serially

⇒ **Serial equivalence**

Interleaving and Serial Schedule

- **Interleaving**
  - A schedule (ordered sequence) of the operations of multiple transactions, where each transaction's operations follow the same order defined by the transaction
- **Serial schedule**
  - Each transaction's operations appear *consecutively* in the schedule *without any intervening operations* from other transactions
- **Example**
  - Let $U = \langle u_1, u_2, u_3 \rangle, \mathrm{V} = \langle v_1, v_2, v_3, v_4 \rangle$
  - $O_1 = \langle u_1, v_1, u_2, v_2, v_3, v_4, u_3 \rangle$ is an interleaving
  - $O_2 = \langle u_1, u_2, u_3, v_1, v_2, v_3, v_4 \rangle$ is a serial schedule
  - $O_3 = \langle u_1, v_1, u_2, v_3, v_2, v_4, u_3 \rangle$ is <u>not</u> an interleaving

not in the same order
as V's original sequence

A Serial Schedule

Transaction T1	Transaction T2	Serial Schedule
x = getSeats(ABC123);		x = getSeats(ABC123);
y = getSeats(ABC789);		y = getSeats(ABC789);
write(x-5, ABC123);		write(x-5, ABC123);
write(y+5, ABC789);		write(y+5, ABC789);
commit		commit
	x = getSeats(ABC123);	x = getSeats(ABC123);
	y = getSeats(ABC789);	y = getSeats(ABC789);
	print("Total:", x+y);	print("Total:", x+y);
	commit	commit

An Interleaving

Transaction T1	Transaction T2	Interleaving
x = getSeats(ABC123);		x = getSeats(ABC123);
y = getSeats(ABC789);		y = getSeats(ABC789);
write(x-5, ABC123);		write(x-5, ABC123);
	x = getSeats(ABC123);	x = getSeats(ABC123);
	y = getSeats(ABC789);	y = getSeats(ABC789);
write(y+5, ABC789);		write(y+5, ABC789);
	print("Total:", x+y);	print("Total:", x+y);
commit		commit
	commit	commit

Serial Equivalence

- **Definition: An interleaving of transaction operations is serially equivalent (or serializable) if its end-result is the same as *some serial schedule* of the same transactions.**
  - In other words, it is not possible to distinguish between the interleaved execution of transaction operations and some serial execution of those operations.

- **Example**
  - Let $U = \langle u_1, u_2, u_3 \rangle, \mathrm{V} = \langle v_1, v_2, v_3, v_4 \rangle$
  - Then, $O = \langle u_1, v_1, u_2, v_2, v_3, v_4, u_3 \rangle$ is serially equivalent if:
  - end-result of O is the same as $\langle u_1, u_2, u_3, v_1, v_2, v_3, v_4 \rangle$; **OR**
  - end-result of O is the same as $\langle v_1, v_2, v_3, v_4, u_1, u_2, u_3 \rangle$

Checking for Serial Equivalence

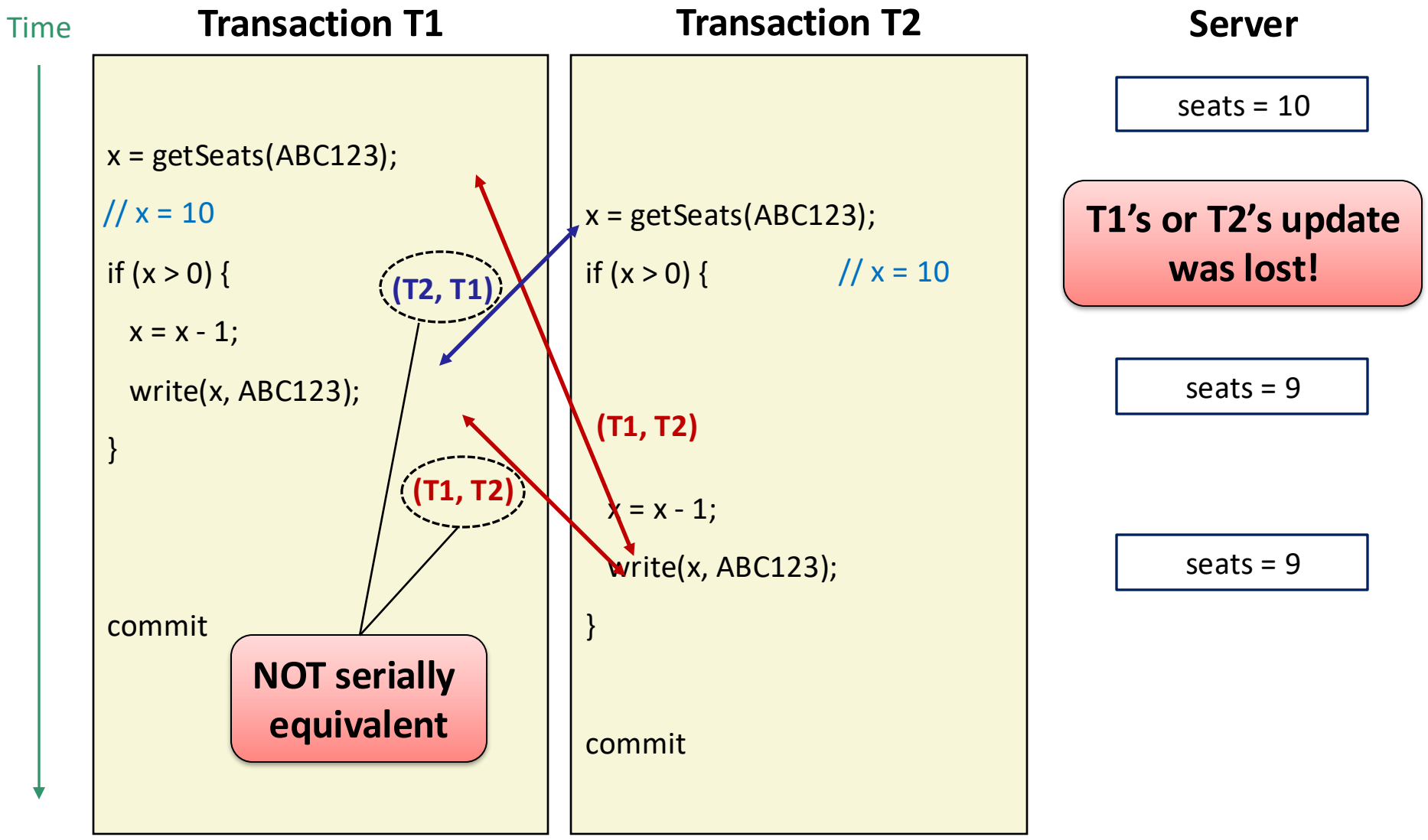
- **An operation has an *effect* on**
  - the server object if it is a write
  - the client (returned value) if it is a read
- **Two operations are said to be conflicting operations if their combined effect depends on the *order* they are executed**
  - read(x) and write(x): conflicting
  - write(x) and read(x): conflicting
  - write(x) and write(x): conflicting
  - read(x) and read(x): not conflicting since swapping them doesn't change their effects
  - read/write(x) and read/write(y): not conflicting since they access different objects

Checking for Serial Equivalence

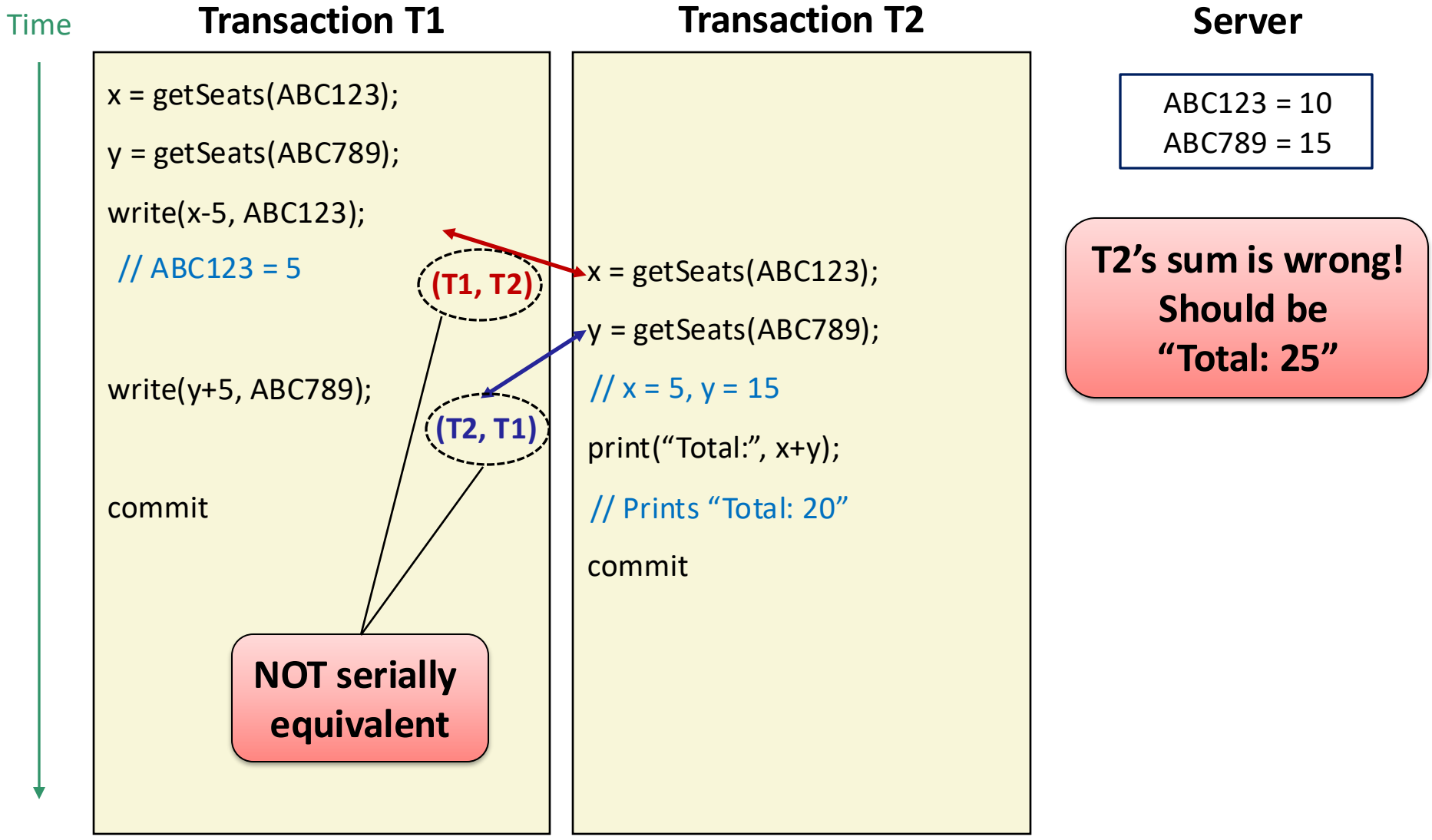
- **An interleaving of two transactions is serially equivalent if and only if all pairs of conflicting operations execute in the same order on all objects they both access**

- ***How do you check for serial equivalence?***
  1. Take all pairs of conflicting operations, one from T1 & one from T2
  2. If the T1 operation is executed first on the server, mark the pair as "(T1, T2)", otherwise mark it as "(T2, T1)"
  3. All pairs should be marked as either "(T1, T2)" or all pairs should be marked as "(T2, T1)"

Lost Update Problem – Caught!



Inconsistent Retrieval Problem – Caught!



Using Serial Equivalence

- **At commit point, coordinator checks transaction T for serial equivalence with all other transactions**
  - Can limit to transactions that overlapped in time with T
- **If not serially equivalent**
  - Abort T
  - Roll back (undo) any writes that T did to server objects
- **But aborting ⇒ wasted work**
- **Can we do better?**

Two Approaches

- **Preventing isolation from being violated can be done in two ways**
  - Pessimistic concurrency control
  - Optimistic concurrency control

- **Pessimistic: assume the worst, prevent transactions from accessing the same object**
  - E.g., Locking

- **Optimistic: assume the best, allow transactions to write, but check later**
  - E.g., Check at commit time, multi-version approaches

Pessimistic: Exclusive Locking

- **Each object has a lock**
  - At most one transaction can be inside lock
- **Before reading or writing object *O*, transaction T must call lock(*O*)**
  - Blocks if another transaction already inside lock
- **After entering lock, T can read and write *O* multiple times**
- **When done (or at commit point), T calls unlock(*O*)**
  - If other transactions waiting at lock(*O*), allows one of them in
- **Sound familiar? (This is Mutual Exclusion!)**

Can We Improve Concurrency?

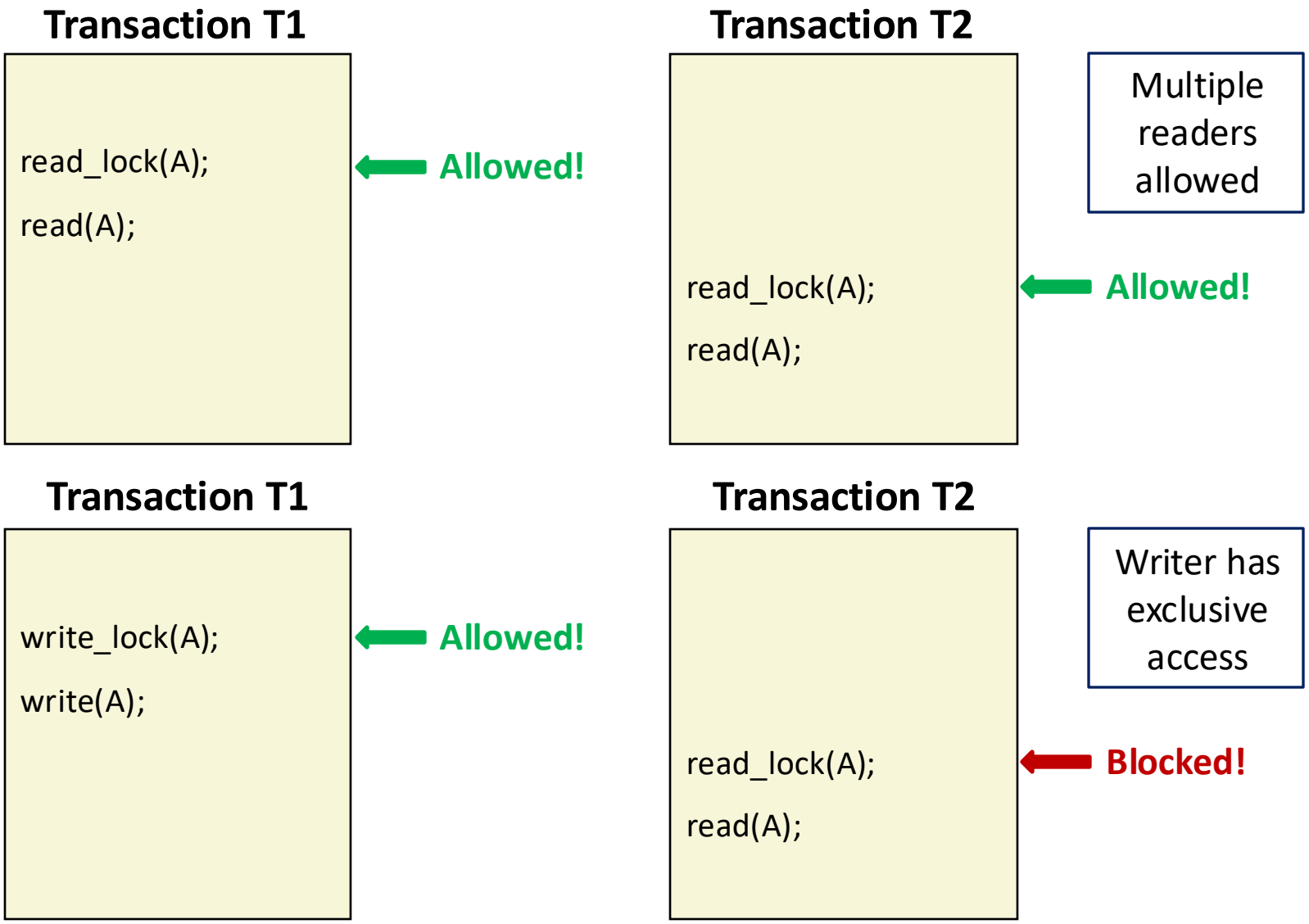
- **More concurrency ⇒ more transactions per second ⇒ more revenue ($$$)**
- **Real-life workloads have a lot of read-only or read-mostly transactions**
  - Exclusive locking reduces concurrency
  - Okay to allow two or more transactions to concurrently read an object, since read-read is not a conflicting pair
- **Improve concurrency by supporting multiple readers**
  - But only one transaction should be able to write to an object and no other transactions should read that data

Read-Write Locks

- **Each object has a lock that can be held in one of two modes**
  - **Read mode:** multiple transactions allowed in this mode
  - **Write mode:** at most one transaction allowed in this mode
- **Basic Idea: allow multiple readers but at most one writer**

- **Before first reading *O*, transaction T calls read_lock(*O*)**
  - T allowed in only if all transactions inside lock for *O* entered via read mode
  - T not allowed to enter (i.e., blocks) if any transaction inside lock for *O* entered via write mode
  - Read lock often called a ***shared lock*** because multiple readers can share it

Example: Read Locks



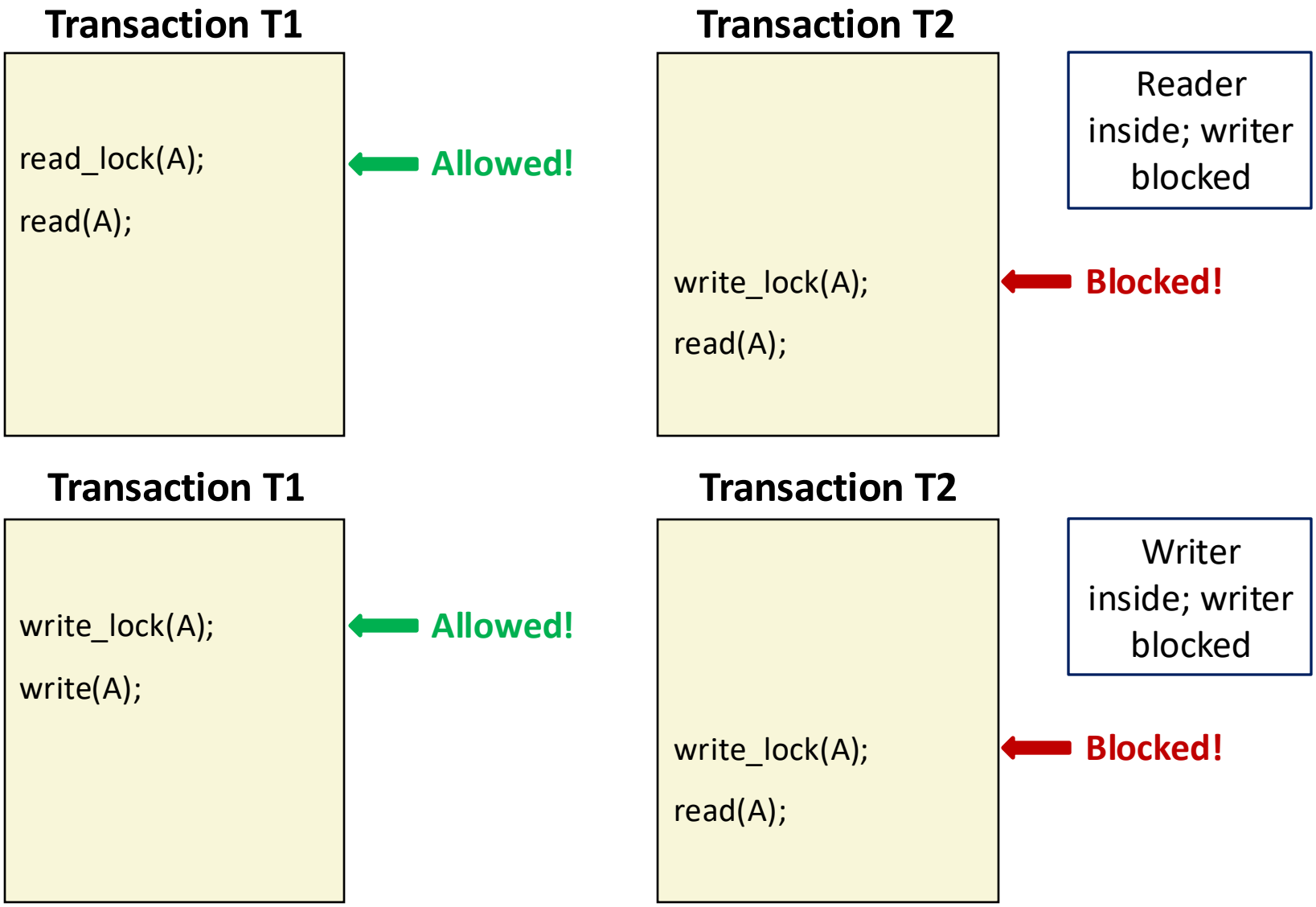
Read-Write Locks, Cont.

- **Before first writing *O*, transaction T calls write_lock(*O*)**
  - T allowed in only if no other transaction inside lock
  - Write lock often called an ***exclusive lock*** because it cannot be shared with anyone (readers or other writers)

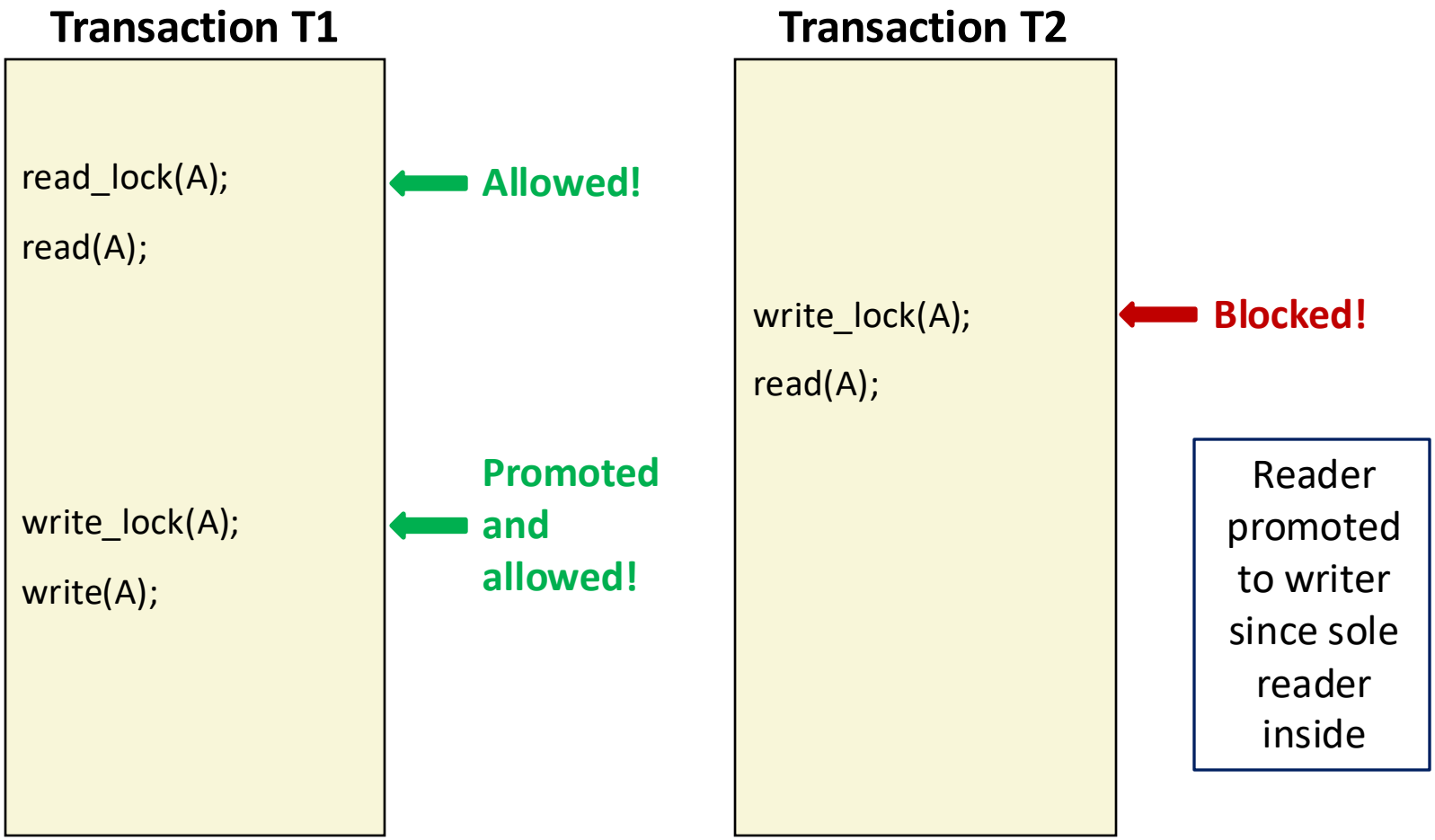
- **If T already holds read_lock(*O*), and wants to write, T calls write_lock(*O*) to *promote* lock from read to write mode**
  - Succeeds only if no other transactions in write mode or read mode
  - Otherwise, T blocks

- **T calls unlock(*O*) to release lock on *O* held by T**

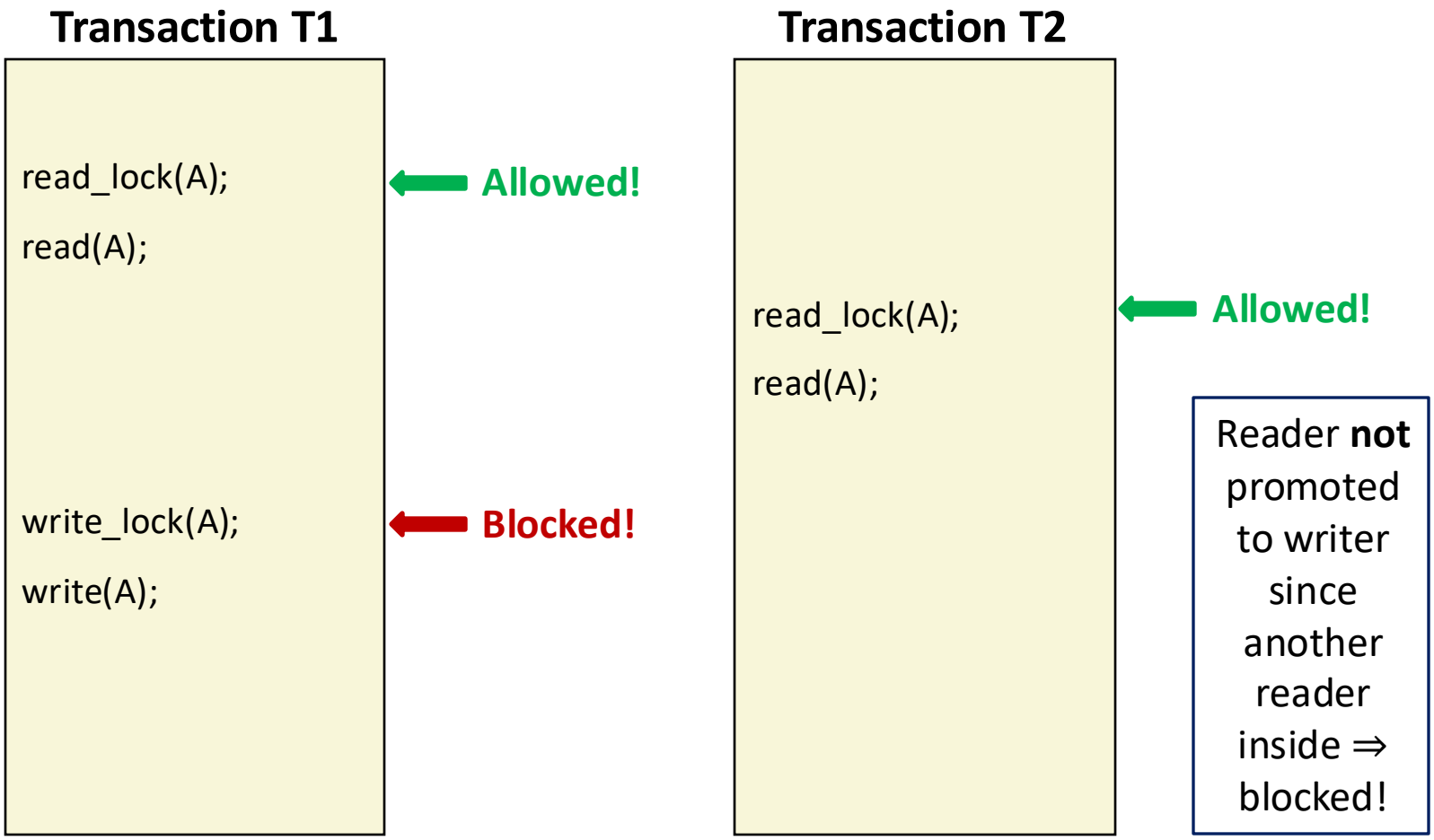
Example: Write Lock



Example: Lock Promotion

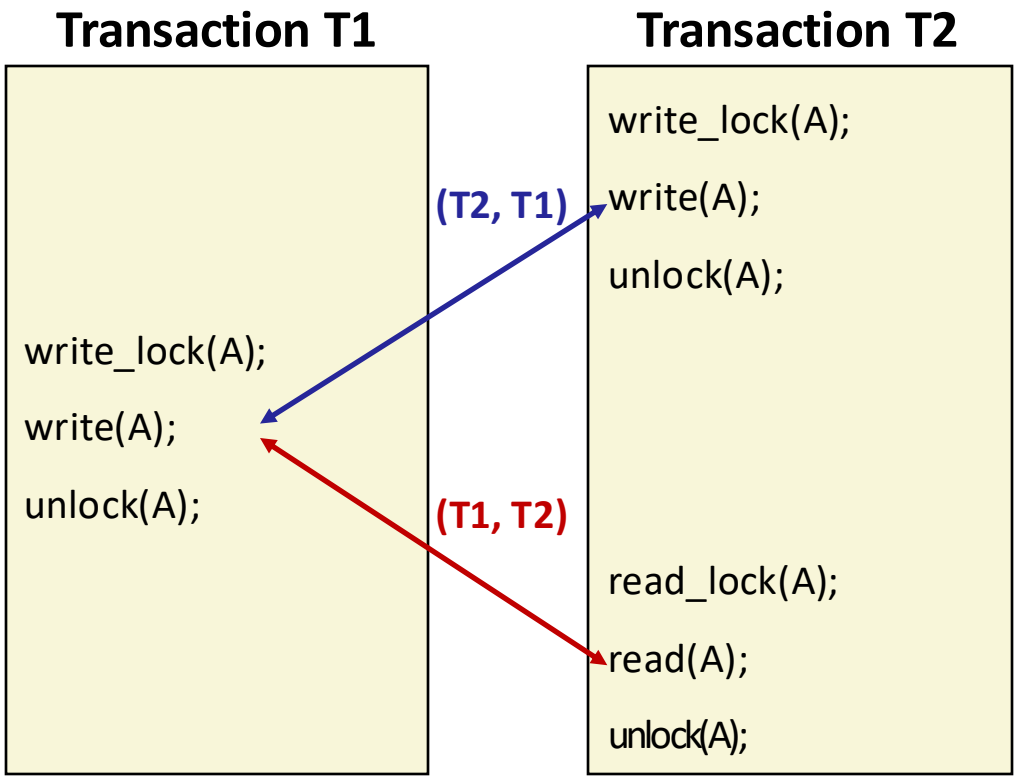


Example: Lock Promotion Fails



When to Release Locks?

- Each transaction tries to acquire lock in the appropriate mode (read or write) to access object
- But when should locks be released?

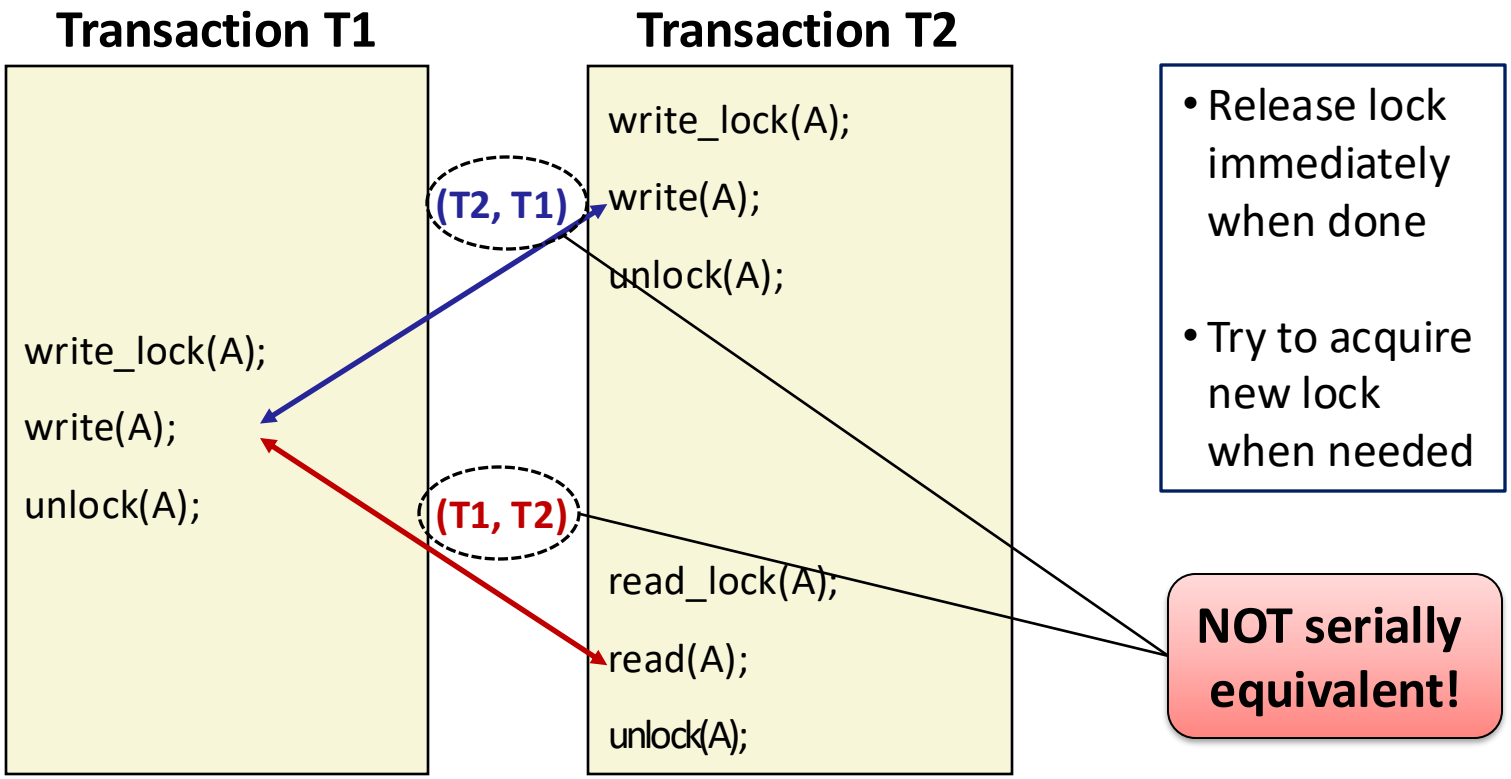


- Release lock immediately when done
- Try to acquire new lock when needed

Is this a good idea?

When to Release Locks?

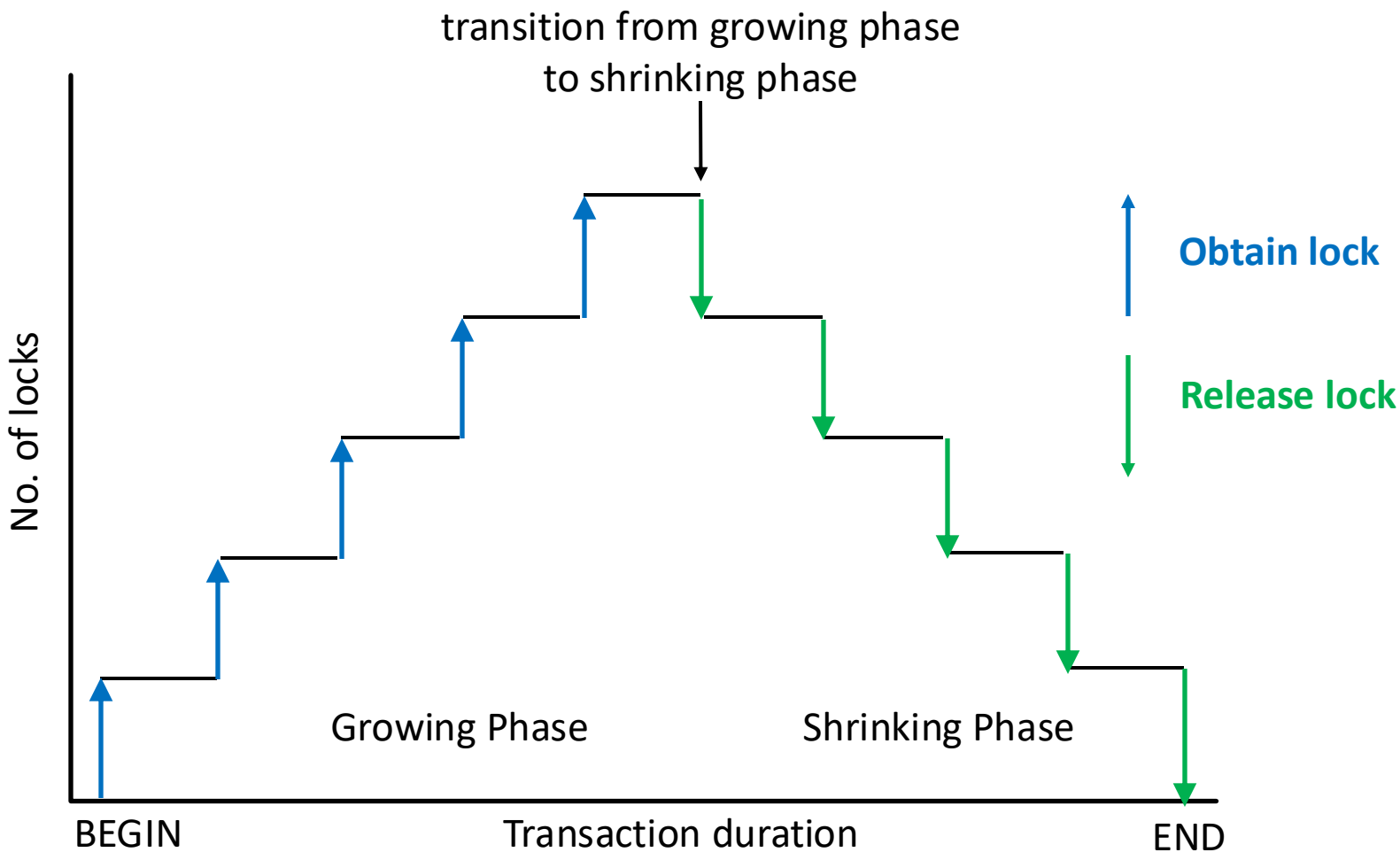
- Each transaction tries to acquire lock in the appropriate mode (read or write) to access object
- But when should locks be released?



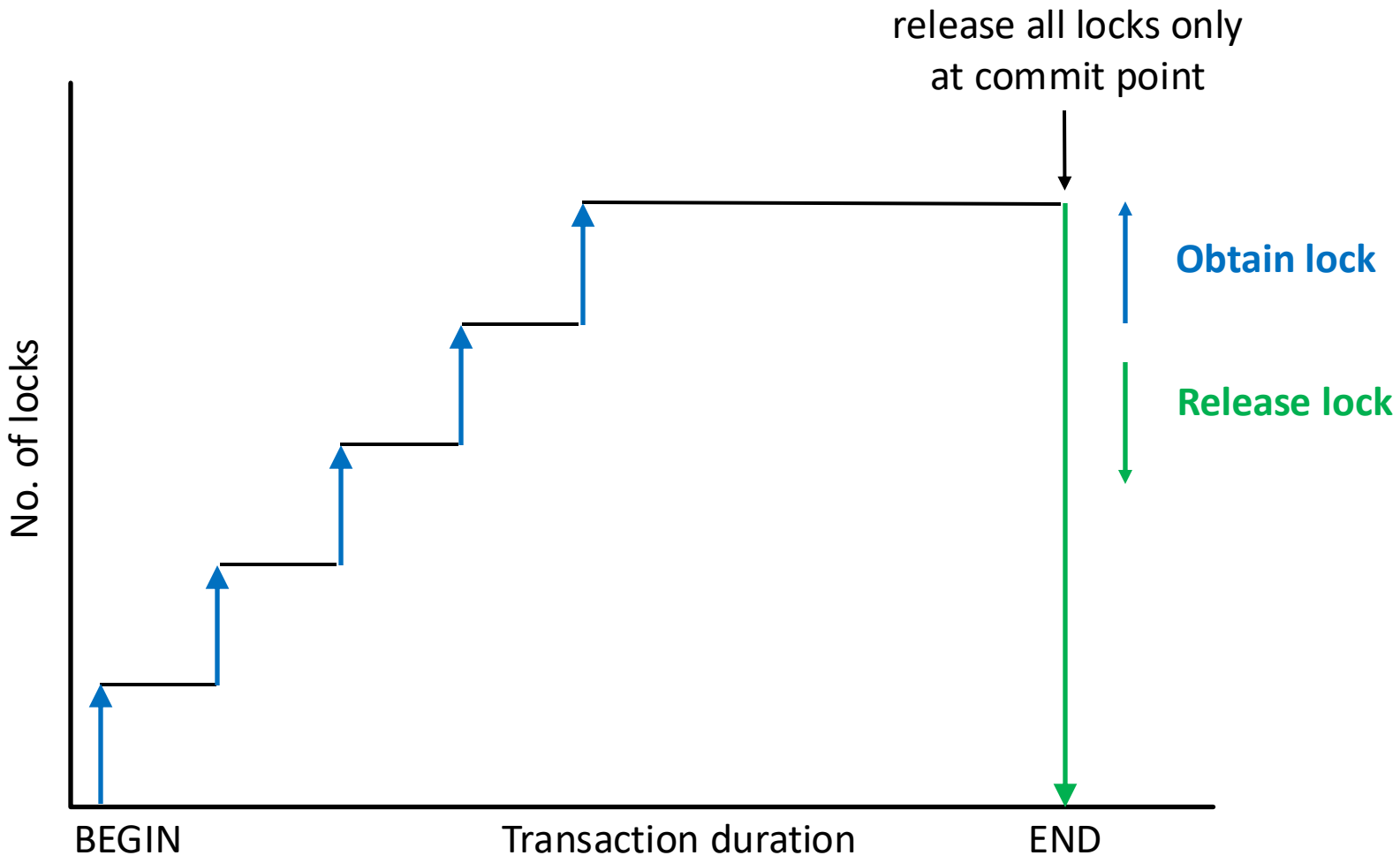
Guaranteeing Serial Equivalence With Locks

- **Two-phase locking**
  - A transaction cannot acquire (or promote) any locks after it has started releasing locks
- **Transaction has two phases**
  - **Growing phase:** only acquires or promotes locks
  - **Shrinking phase:** only releases locks
- **Strict two-phase locking: releases locks only at commit point**

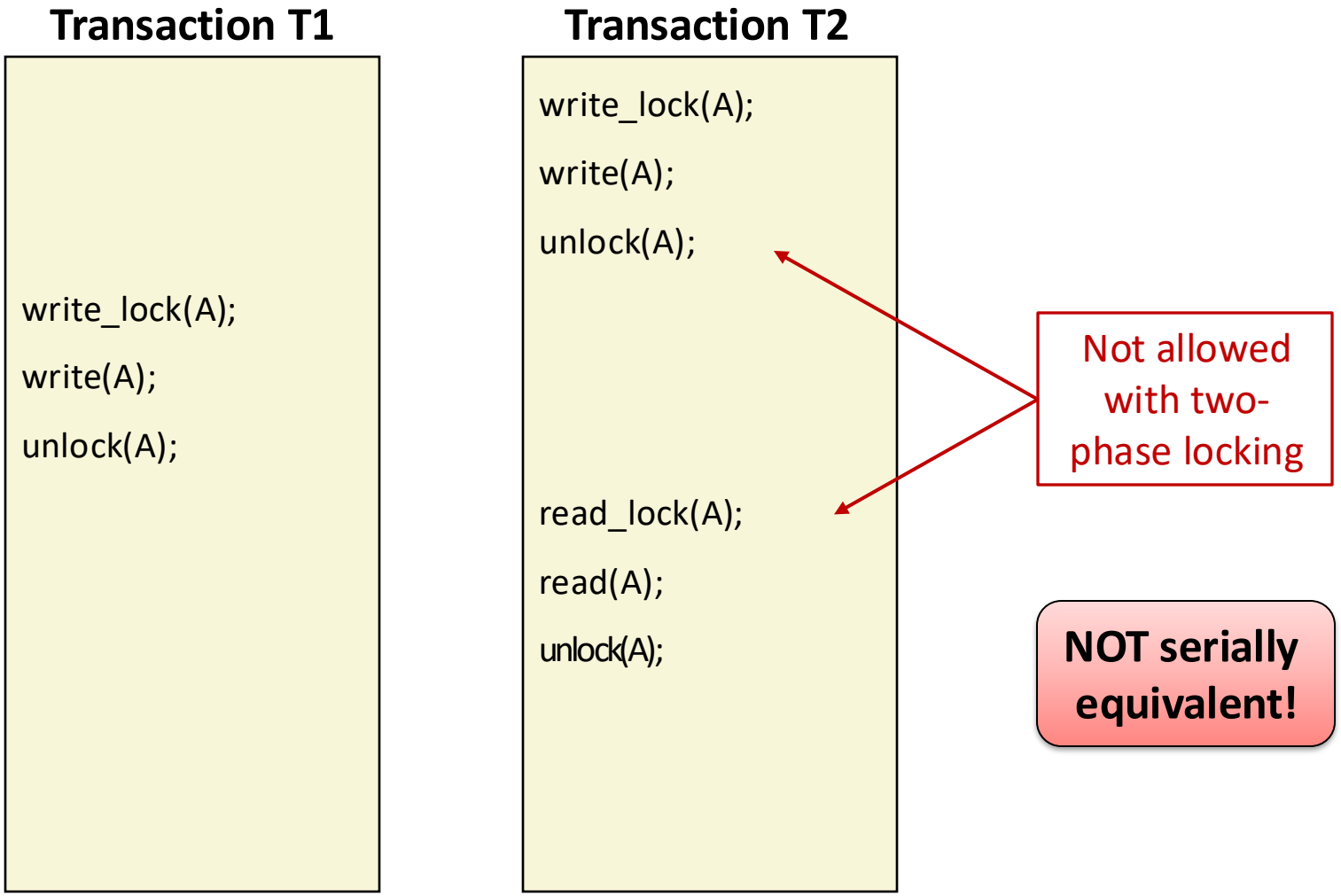
Two-Phase Locking



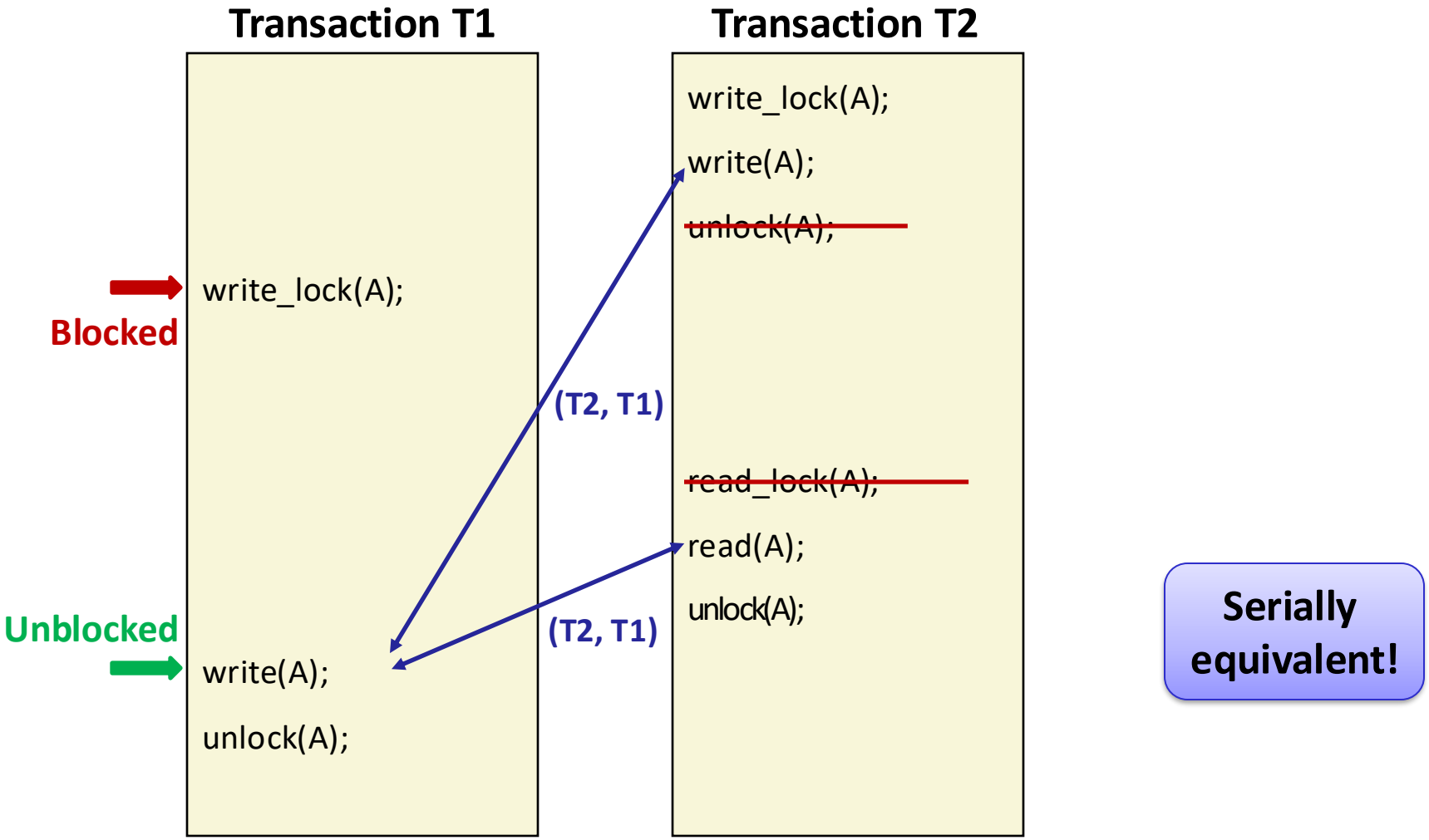
Strict Two-Phase Locking



Two-Phase Locking



Two-Phase Locking



Serially equivalent!

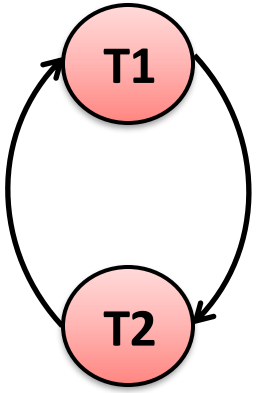
Why Two-Phase Locking ⇒ Serial Equivalence

- **Proof by contradiction**
- **Assume two-phase locking system where serial equivalence is violated for some two transactions T1, T2**
- **Two facts must then be true:**
  - (A) For some object *O*1, there were conflicting operations in T1 and T2 such that the time ordering pair is (T1, T2)
  - (B) For some object *O*2, the conflicting operation pair is (T2, T1)
- **Contradiction arises from the following:**
  - (A) ⇒ T1 released *O*1's lock and T2 acquired it after that ⇒ T1's shrinking phase is before or overlaps with T2's growing phase
  - Similarly, (B) ⇒ T2's shrinking phase is before or overlaps with T1's growing phase
  - But both these cannot be true! ⇒ Contradiction.

Downside of Locking – Deadlock!

Time

Transaction T1	Transaction T2
Lock(ABC123);	Lock(ABC789);
x = write(10,ABC123);	y = write(15,ABC789);
Lock(ABC789);	Lock(ABC123);
// Blocks waiting for T2	// Blocks waiting for T1
…	…



Wait-For Graph

Combating Deadlocks

1. **Lock timeout**
   - Abort transaction if lock cannot be acquired within timeout
   - Disadvantage: expensive; leads to wasted work
   - Also, how to determine timeout value?
     - *Too large ⇒ long delays*
     - *Too small ⇒ false positives*

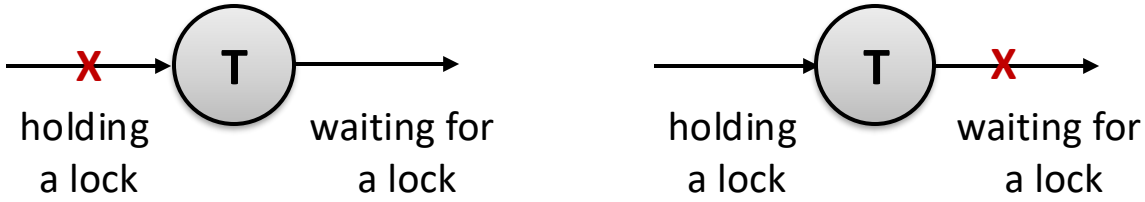
2. **Deadlock Detection**
   - Keep track of wait-for graph and find cycles in it (e.g., periodically)
   - If find cycle, there's a deadlock

     ⇒ Abort one or more transactions to break cycle
   - Disadvantage: still allows deadlocks to occur

Combating Deadlocks, Cont.

3. Deadlock Prevention

- Prevent cycles in the wait-for graph from occurring
- Each transaction must request all locks it needs at the beginning; blocks until all locks are acquired simultaneously
  - *Can't have a cycle in the wait-for graph since a transaction cannot simultaneously hold a lock and wait for another*



- Disadvantages:
  - *Low resource utilization: a lock may be held for a long time before object is actually accessed*
  - *Starvation: a transaction may have to wait indefinitely for all locks it requested to become available*

Problems with Locking

- **Locks have an overhead: maintenance, checking**
- **Locks can result in deadlock**
- **Locks may reduce concurrency**
  - Transactions hold the locks until the transaction commits (strong strict two-phase locking)
- **But … if data is not locked**
  - A transaction may see inconsistent results
  - Locking solves this problem … but incurs delays

Optimistic Concurrency Control

- Increases concurrency more than pessimistic concurrency control
- Increases transactions per second
- For non-transaction systems, increases operations per second and lowers latency
- Used in Dropbox, Google apps, Wikipedia, key-value stores like Cassandra, Riak, and Amazon's Dynamo
- Preferable than pessimistic when conflicts are expected to be rare
  - But still need to ensure conflicts are caught!

First-Cut Approach

- **Most basic approach**
  - Write and read objects at will
  - Check for serial equivalence at commit time
  - If abort, roll back updates made
  - An abort may result in other transactions that read dirty data, also being aborted …
  - Any transactions that read from *those* transactions also now need to be aborted …

☹ ***Cascading aborts***

Timestamp Ordering

- **Each transaction is assigned a unique timestamp when it starts**
  - A transaction's timestamp defines its position in the time sequence of transactions
- **Ensure that the following timestamp ordering rule is followed:**
  1. A transaction's request to write an object is valid only if that object was last read or written by earlier transactions
  2. A transaction's request to read an object is valid only if that object was last written by an earlier transaction
- **Implemented by maintaining read and write timestamps for the object**
- **If rule is violated, abort transaction**
- **Never results in deadlock – older transaction never waits on newer ones**

Timestamp Ordering Implementation

- **Transaction timestamp**
  - Transaction IDs are assigned in increasing order
  - Transaction ID will serve as transaction's timestamp
  - E.g., $T_i < T_j$ means T_i's timestamp < T_j's timestamp
- **Per-object state**
  - Committed value
  - Transaction ID that wrote the committed value
  - Read timestamps (*RTS*): list of transaction IDs that have read the committed value
  - Tentative writes (*TW*): list of tentative writes sorted by increasing transaction IDs
    - *These serve as timestamped versions of the object*

Timestamp Ordering Rules

- **A request by a transaction T_c for a read or write operation on an object must conform to the following rules:**

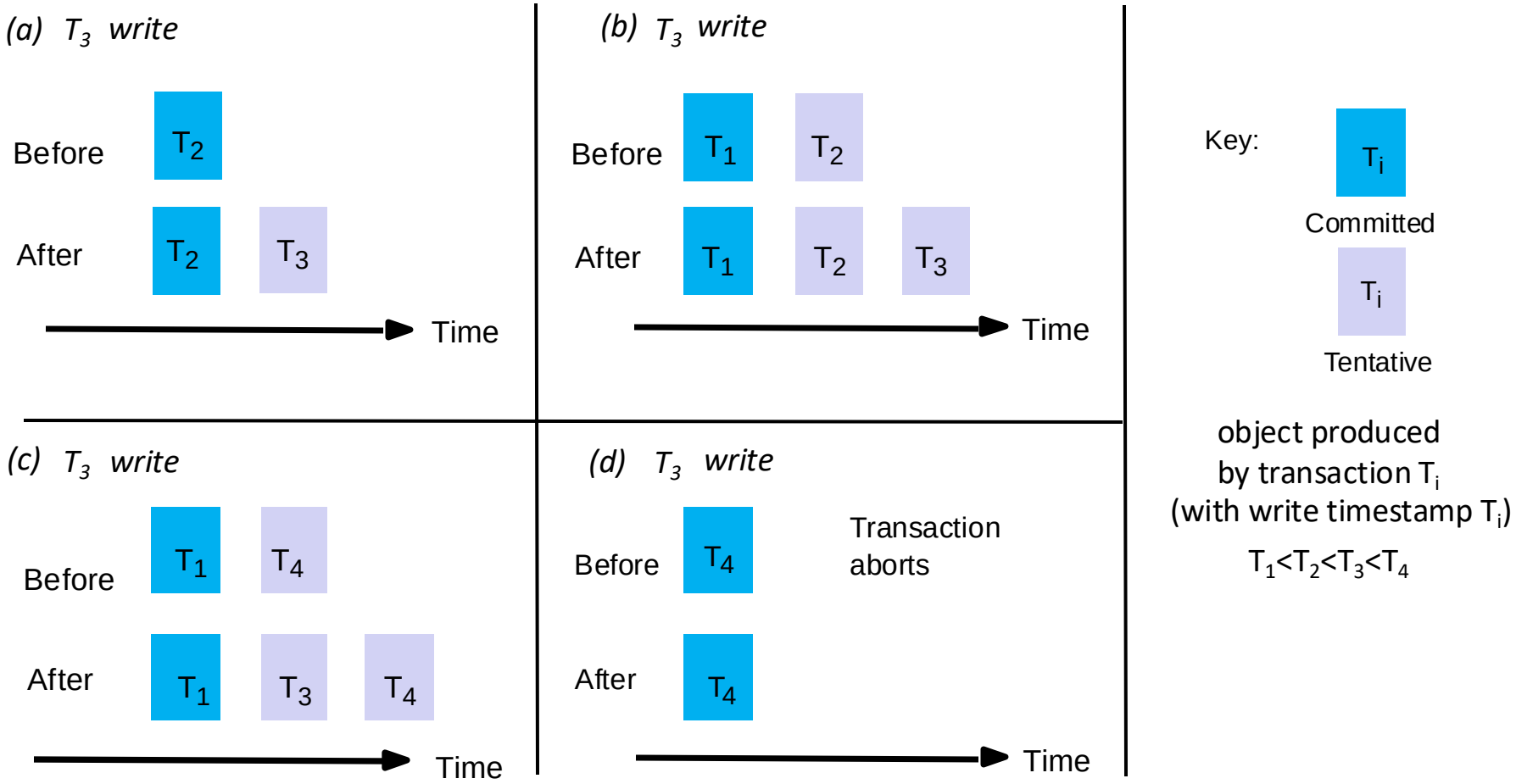
	T_c	T_i	Rule
1.	write	read	T_c must not write an object that has been read by any $T_i > T_c$. This requires that $T_c \geq$ maximum read timestamp of the object.
2.	write	write	T_c must not write an object that has been written by any $T_i > T_c$. This requires that $T_c >$ write timestamp of the committed object.
3.	read	write	T_c must not read an object that has been written by any $T_i > T_c$. This requires that $T_c >$ write timestamp of the committed object.

Timestamp Ordering: Write Rule

- **Transaction T_c requests a write operation on object *D***

```
if (Tc ≥ maximum read timestamp on D
    &&  Tc > write timestamp on committed version of D) {
        perform a tentative write on D
        if Tc already has an entry in the TW list for D, update it
        else add Tc and its write value to the TW list
}
else
        abort transaction Tc
        // too late: a transaction with later timestamp has
           already read or written object D
```

Illustration of Write Rule



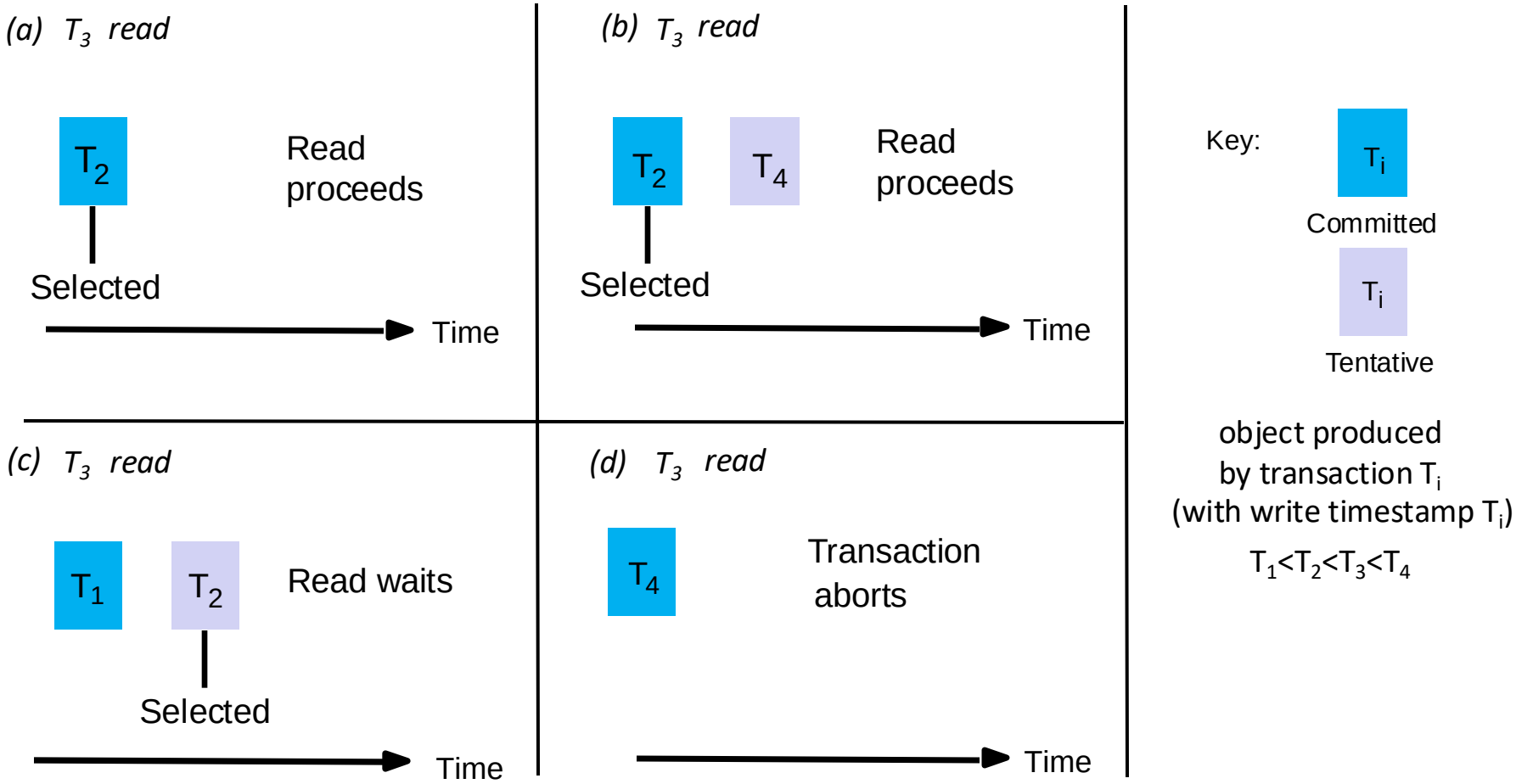
Assume that $T_3 \geq$ maximum read timestamp on object (not shown)

Timestamp Ordering: Read Rule

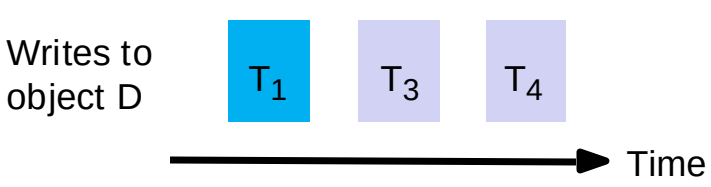
- **Transaction T_c requests a read operation on object D**

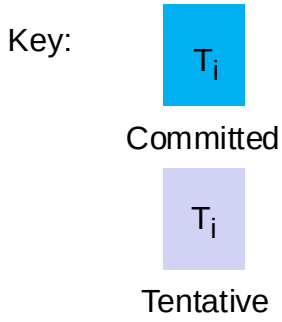
```
if (Tc > write timestamp on committed version of D) {
    D_selected = version of D with the maximum write timestamp ≤ Tc
    if (D_selected is committed)
        read D_selected and add Tc to RTS list of D (if not already added)
    elseif (D_selected was written by Tc)
        simply read D_selected
    else
        wait until the transaction that wrote D_selected is committed or aborted,
        and reapply the read rule
        // if transaction is committed, Tc will read its value after the wait
        // if transaction is aborted, Tc will read the value from an older transaction
}
else
    abort transaction Tc
    // too late; a transaction with a later timestamp has already written object D
```

Illustration of Read Rule



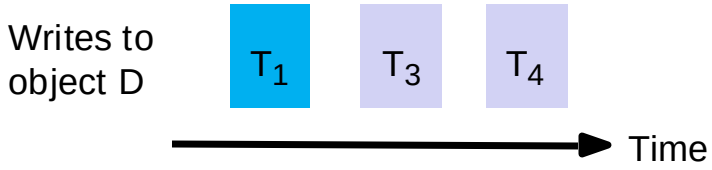
Timestamp Ordering: Committing

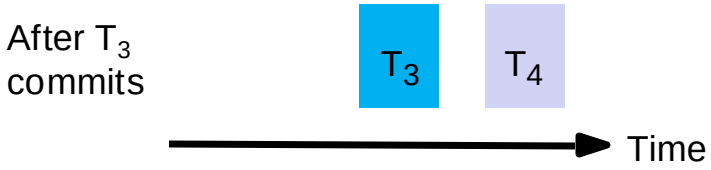




- Suppose T_4 is ready to commit
- T_4 must wait until T_3 commits or aborts

- When a transaction is committed:
  - Committed value of object and associated timestamp are updated
  - Corresponding write removed from *TW* list





Lost Update Example with Timestamp Ordering

Transaction T1

```
x = getSeats(ABC123);

if (x > 0) {
  x = x - 1;
  write(x, ABC123);
}


commit
```

Transaction T2

```
x = getSeats(ABC123);
if (x > 0) {




  x = x - 1;
  write(x, ABC123);
}


commit
```

ABC123 state:
committed value = **10**
committed timestamp = **0**
RTS:
TW:

Lost Update Example with Timestamp Ordering

Transaction T1

```
x = getSeats(ABC123);

if (x > 0) {
  x = x - 1;
  write(x, ABC123);
}

commit
```

Transaction T2

```
x = getSeats(ABC123);
if (x > 0) {

  x = x - 1;
  write(x, ABC123);
}

commit
```

ABC123 state:
committed value = **10**
committed timestamp = **0**
RTS: **1**
TW:

Lost Update Example with Timestamp Ordering

Transaction T1

```
x = getSeats(ABC123);

if (x > 0) {
  x = x - 1;
  write(x, ABC123);
}

commit
```

Transaction T2

```
x = getSeats(ABC123);
if (x > 0) {

  x = x - 1;
  write(x, ABC123);
}

commit
```

ABC123 state:
committed value = **10**
committed timestamp = **0**
RTS: **1, 2**
TW:

Lost Update Example with Timestamp Ordering

Transaction T1

```
x = getSeats(ABC123);

if (x > 0) {
  x = x - 1;
  write(x, ABC123);
}

commit
```

Transaction T2

```
x = getSeats(ABC123);
if (x > 0) {

  x = x - 1;
  write(x, ABC123);
}

commit
```

ABC123 state:
committed value = **10**
committed timestamp = **0**
RTS: **1, 2**
TW:

$T_1 \not\geq$ max. read timestamp of ABC123 (=2)

Abort T1!

Inconsistent Retrieval Example with Timestamp Ordering

Transaction T1

```
x = getSeats(ABC123);
y = getSeats(ABC789);
write(x-5, ABC123);



write(y+5, ABC789);

commit
```

Transaction T2

```
x = getSeats(ABC123);
y = getSeats(ABC789);

print("Total:", x+y);

commit
```

ABC123 state:
committed value = **10**
committed timestamp = **0**
RTS:
TW:

ABC789 state:
committed value = **15**
committed timestamp = **0**
RTS:
TW:

Inconsistent Retrieval Example with Timestamp Ordering

Transaction T1

```
x = getSeats(ABC123);
y = getSeats(ABC789);
write(x-5, ABC123);



write(y+5, ABC789);

commit
```

Transaction T2

```
x = getSeats(ABC123);
y = getSeats(ABC789);

print("Total:", x+y);

commit
```

ABC123 state:
committed value = **10**
committed timestamp = **0**
RTS: **1**
TW:

ABC789 state:
committed value = **15**
committed timestamp = **0**
RTS:
TW:

Inconsistent Retrieval Example with Timestamp Ordering

Transaction T1

```
x = getSeats(ABC123);
y = getSeats(ABC789);
write(x-5, ABC123);


write(y+5, ABC789);

commit
```

Transaction T2

```
x = getSeats(ABC123);
y = getSeats(ABC789);

print("Total:", x+y);

commit
```

ABC123 state:
committed value = **10**
committed timestamp = **0**
RTS: **1**
TW:

ABC789 state:
committed value = **15**
committed timestamp = **0**
RTS: **1**
TW:

Inconsistent Retrieval Example with Timestamp Ordering

Transaction T1

```
x = getSeats(ABC123);
y = getSeats(ABC789);
write(x-5, ABC123);


write(y+5, ABC789);

commit
```

Transaction T2

```
x = getSeats(ABC123);
y = getSeats(ABC789);

print("Total:", x+y);

commit
```

ABC123 state:
committed value = **10**
committed timestamp = **0**
RTS: **1**
TW: **(5,1)**

ABC789 state:
committed value = **15**
committed timestamp = **0**
RTS: **1**
TW:

Inconsistent Retrieval Example with Timestamp Ordering

Transaction T1

```
x = getSeats(ABC123);
y = getSeats(ABC789);
write(x-5, ABC123);



write(y+5, ABC789);

commit
```

Transaction T2

```
x = getSeats(ABC123);
y = getSeats(ABC789);

print("Total:", x+y);

commit
```

ABC123 state:
committed value = **10**
committed timestamp = **0**
RTS: **1**
TW: **(5,1)**

Version of ABC123 with maximum timestamp is the tentative write by T_1
$\Rightarrow$ wait for T_1

ABC789 state:
committed value = **15**
committed timestamp = **0**
RTS: **1**
TW:

Inconsistent Retrieval Example with Timestamp Ordering

Transaction T1

```
x = getSeats(ABC123);
y = getSeats(ABC789);
write(x-5, ABC123);

write(y+5, ABC789);

commit
```

wait

Transaction T2

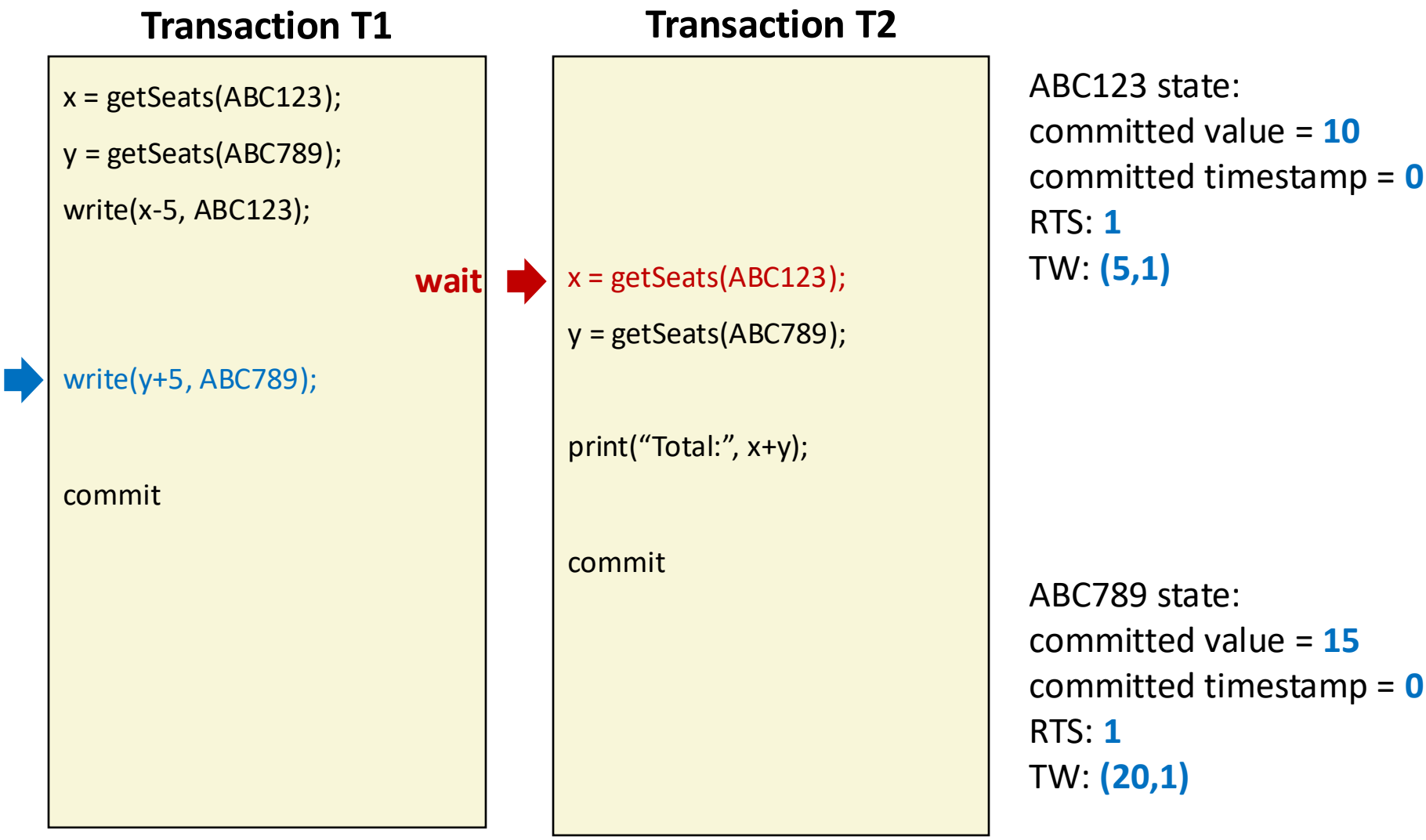
```
x = getSeats(ABC123);
y = getSeats(ABC789);

print("Total:", x+y);

commit
```

ABC123 state:
committed value = **10**
committed timestamp = **0**
RTS: **1**
TW: **(5,1)**

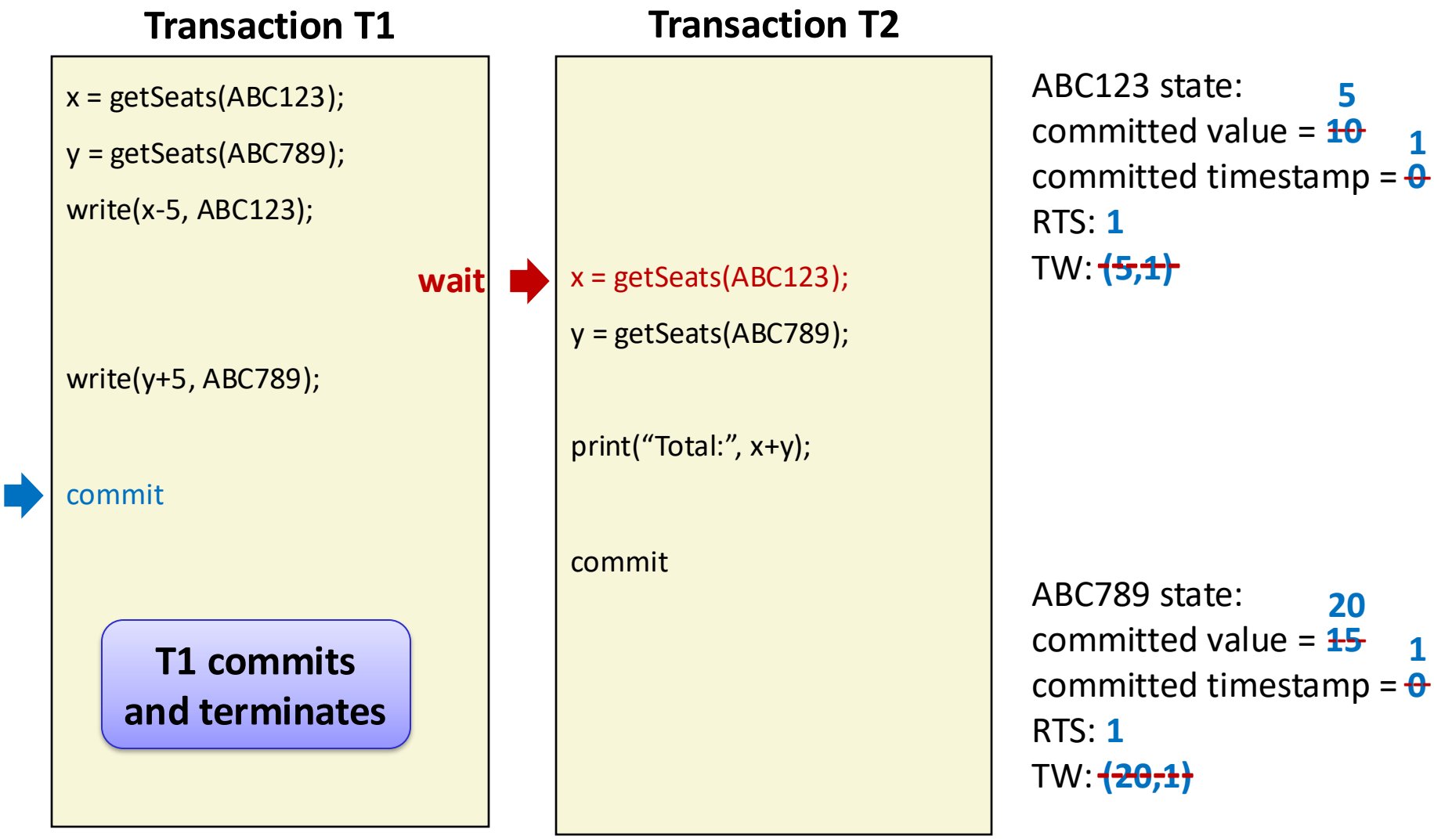
Version of ABC123 with maximum timestamp is the tentative write by T_1
$\Rightarrow$ wait for T_1

ABC789 state:
committed value = **15**
committed timestamp = **0**
RTS: **1**
TW:

Inconsistent Retrieval Example with Timestamp Ordering



Inconsistent Retrieval Example with Timestamp Ordering



Inconsistent Retrieval Example with Timestamp Ordering

Transaction T1

```
x = getSeats(ABC123);
y = getSeats(ABC789);
write(x-5, ABC123);


write(y+5, ABC789);

commit
```

Transaction T2

```
x = getSeats(ABC123);
y = getSeats(ABC789);

print("Total:", x+y);

commit
```

T2 proceeds after T1

ABC123 state:
committed value = **5**
committed timestamp = **1**
RTS: **1, 2**
TW:

T2 reads x = 5

ABC789 state:
committed value = **20**
committed timestamp = **1**
RTS: **1**
TW:

Inconsistent Retrieval Example with Timestamp Ordering

Transaction T1

```
x = getSeats(ABC123);
y = getSeats(ABC789);
write(x-5, ABC123);



write(y+5, ABC789);

commit
```

Transaction T2

```
x = getSeats(ABC123);
y = getSeats(ABC789);

print("Total:", x+y);

commit
```

T2 proceeds after T1

ABC123 state:
committed value = **5**
committed timestamp = **1**
RTS: **1, 2**
TW:

T2 reads x = 5
T2 reads y = 20

ABC789 state:
committed value = **20**
committed timestamp = **1**
RTS: **1, 2**
TW:

Inconsistent Retrieval Example with Timestamp Ordering

Transaction T1

```
x = getSeats(ABC123);
y = getSeats(ABC789);
write(x-5, ABC123);


write(y+5, ABC789);

commit
```

Transaction T2

```
x = getSeats(ABC123);
y = getSeats(ABC789);

print("Total:", x+y);

commit
```

T2 proceeds after T1

ABC123 state:
committed value = **5**
committed timestamp = **1**
RTS: **1, 2**
TW:

T2 reads x = 5
T2 reads y = 20
T2 prints "Total: 25"

ABC789 state:
committed value = **20**
committed timestamp = **1**
RTS: **1, 2**
TW:

Inconsistent Retrieval Example with Timestamp Ordering

Transaction T1

```
x = getSeats(ABC123);
y = getSeats(ABC789);
write(x-5, ABC123);


write(y+5, ABC789);

commit
```

Transaction T2

```
x = getSeats(ABC123);
y = getSeats(ABC789);

print("Total:", x+y);

commit
```

T2 commits and terminates

ABC123 state:
committed value = **5**
committed timestamp = **1**
RTS: **1, 2**
TW:

T2 reads x = 5
T2 reads y = 20
T2 prints "Total: 25"

ABC789 state:
committed value = **20**
committed timestamp = **1**
RTS: **1, 2**
TW:

Summary

- **Transactions**
- **Serial Equivalence**
  - Detecting it via conflicting operations
- **Pessimistic Concurrency Control**
  - Read-write locks with two-phase locking and deadlock detection
- **Optimistic Concurrency Control**
  - Timestamp ordering

The End

Credits:

This lecture contains slides originally developed by Indranil Gupta and Radhika Mittal of UIUC